Part II: Forward Curves & Multi-Commodity Modeling
2.3 State-Space Models and the Kalman Filter

Commodity Markets and Models

ESILV

J. Chevallier

Outline

1. 9.1 State-Space Representations
2. 9.2 The Basic Structural Model
3. 9.3 State-Space Representation of ARIMA Models
4. 9.4 The Kalman Recursions
5. 9.5 Estimation for State-Space Models
6. 9.6 State-Space Models with Missing Observations
7. 9.7 The EM Algorithm
8. Application to Commodity Forward Curves

9.1 State-Space Representations: Basic Formulation

Definition 9.1.1 (State-Space Model)

A state-space model for a time series $\{\mathbf{Y}_t\}$ consists of two fundamental equations:

$$\textbf{Observation equation:} \quad \mathbf{Y}_t = G_t \mathbf{X}_t + \mathbf{W}_t, \quad t = 1, 2, \ldots \tag{9.1.1}$$

$$\textbf{State equation:} \quad \mathbf{X}_{t+1} = F_t \mathbf{X}_t + \mathbf{V}_t, \quad t = 1, 2, \ldots \tag{9.1.2}$$

where:

- $\{\mathbf{W}_t\} \sim \mathrm{WN}(\mathbf{0}, \{R_t\})$ is **measurement noise**.
- $\{\mathbf{V}_t\} \sim \mathrm{WN}(\mathbf{0}, \{Q_t\})$ is **process noise** (system shocks).
- Initial state $\mathbf{X}_1$ is uncorrelated with all noise terms.

Matrix Dimensions (Pedagogical Note)

- $\mathbf{X}_t$: $v \times 1$ **state vector** (The hidden "truth").
- $\mathbf{Y}_t$: $w \times 1$ **observation vector** (The noisy market data).
- F_t $(v \times v)$: How the state evolves.
- G_t $(w \times v)$: How the state maps to observations.

Remarks and Special Cases

Remarks from Section 9.1

- **Remark 1**: General forms allow $\mathrm{Cov}(\mathbf{V}_t, \mathbf{W}_t) = S_t$.
- **Remark 2**: **Time-invariant** systems: $F_t = F, G_t = G, Q_t = Q, R_t = R$.
- **Remark 3**: Functional forms from (9.1.3)-(9.1.4):

$$\mathbf{X}_t = f_t(\mathbf{X}_1, \mathbf{V}_1, \ldots, \mathbf{V}_{t-1}), \quad \mathbf{Y}_t = g_t(\mathbf{X}_1, \mathbf{V}_1, \ldots, \mathbf{V}_{t-1}, \mathbf{W}_t)$$

- **Remark 4**: Orthogonality ensures we obtain the **Best Linear Predictor**.

Pedagogical Takeaway

The state $\mathbf{X}_t$ is a **sufficient statistic** for the future: once we know the current state, the past history of observations provides no additional predictive value.

Examples: AR(1) and ARMA(1,1) Processes

Example 9.1.1: AR(1) Process

For $Y_t = \phi Y_{t-1} + Z_t$, $Z_t \sim \mathrm{WN}(0, \sigma^2)$:

$$\begin{aligned} \text{State equation:} \quad & X_{t+1} = \phi X_t + V_t, \quad V_t = Z_{t+1} \\ \text{Observation equation:} \quad & Y_t = X_t \end{aligned}$$

Parameters: $G_t = 1$, $W_t = 0$, $F_t = \phi$, $Q_t = \sigma^2$, $R_t = 0$.

Example 9.1.2: ARMA(1,1) Process

For $Y_t = \phi Y_{t-1} + Z_t + \theta Z_{t-1}$:

$$\text{Let } X_t \text{ satisfy } \phi(B) X_t = Z_t$$
$$\text{Then } Y_t = \theta(B) X_t = \theta X_{t-1} + X_t$$

State vector: $\mathbf{X}_t = (X_{t-1}, X_t)'$. This demonstrates that classical ARMA models are just restricted cases of the state-space framework.

9.2.1 Random Walk Plus Noise Model

Local Level Model (Equations 9.2.1-9.2.2)

Models a latent equilibrium moving as a random walk:

$$Y_t = M_t + W_t, \quad \{W_t\} \sim \mathrm{WN}(0, \sigma_w^2) \tag{9.2.1}$$

$$M_{t+1} = M_t + V_t, \quad \{V_t\} \sim \mathrm{WN}(0, \sigma_v^2) \tag{9.2.2}$$

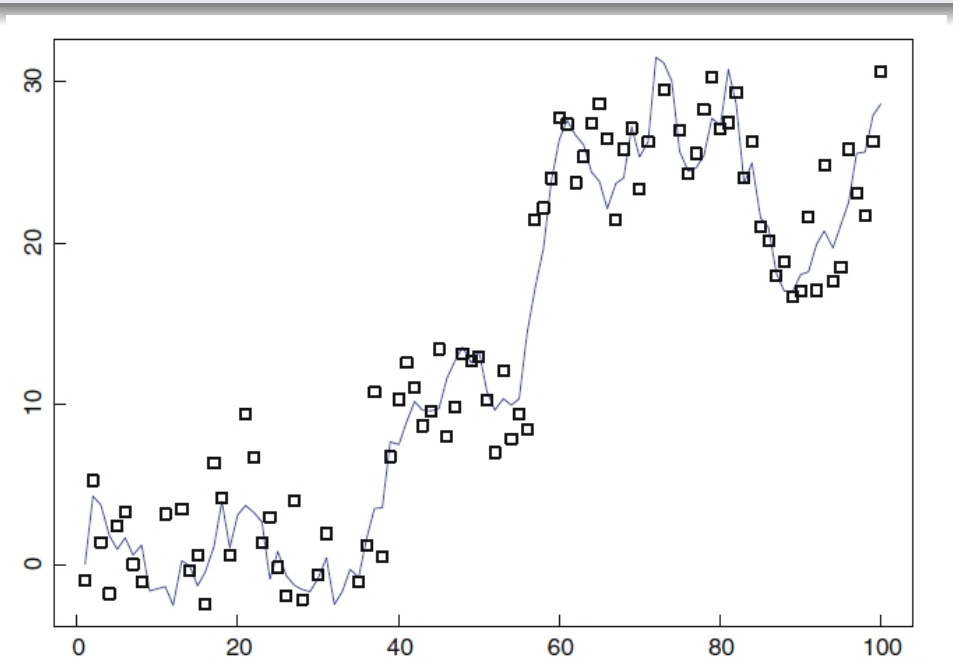


Figure: 9-1: The random walk (solid line) and observed data (boxes).

Local Linear Trend Model

Equations (9.2.4)-(9.2.7)

Allows the trend level (M_t) and slope (B_t) to evolve:

$$M_t = M_{t-1} + B_{t-1} + V_{t-1} \tag{9.2.4}$$

$$B_t = B_{t-1} + U_{t-1}, \quad \{U_t\} \sim \mathrm{WN}(0, \sigma_u^2) \tag{9.2.5}$$

$$\mathbf{X}_{t+1} = \begin{bmatrix} 1 & 1 \\ 0 & 1 \end{bmatrix} \mathbf{X}_t + \mathbf{V}_t, \quad \mathbf{X}_t = (M_t, B_t)' \tag{9.2.6}$$

$$Y_t = [1 \quad 0]\mathbf{X}_t + W_t \tag{9.2.7}$$

Application Insight

In commodity markets, M_t can represent the long-term equilibrium price, while B_t captures evolving **convenience yields** or storage costs.

Seasonal Components

Seasonal Model (Equations 9.2.8-9.2.12)

For period d, seasonal components must sum to zero over a cycle:

$$s_{t+1} = -s_t - \cdots - s_{t-d+2} \tag{9.2.8}$$

$$Y_{t+1} = -Y_t - \cdots - Y_{t-d+2} + S_t \quad \text{(with noise)} \tag{9.2.9}$$

State-space representation with state vector $\mathbf{X}_t = (Y_t, Y_{t-1}, \ldots, Y_{t-d+2})'$:

$$\mathbf{X}_{t+1} = F\mathbf{X}_t + \mathbf{V}_t, \quad F = \begin{bmatrix} -1 & -1 & \cdots & -1 \\ 1 & 0 & \cdots & 0 \\ \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \cdots & 0 \end{bmatrix} \tag{9.2.11, 9.2.12}$$

Why Put ARIMA in State-Space Form?

The Big Idea

Any ARIMA model can be rewritten as a **state-space model**. This lets us:

- Use the **Kalman filter** for efficient estimation and forecasting
- Handle **missing data** naturally
- Compute **exact likelihood** for parameter estimation
- Forecast **multiple steps ahead** recursively

Key Insight

ARIMA models are **scalar** equations (relating Y_t to its past).
State-space form turns them into **vector** equations (first-order Markov process).
This transformation makes computation and theory much cleaner!

AR(2) Example: From Scalar to Vector Form

Original AR(2) Model

$$Y_t = \phi_1 Y_{t-1} + \phi_2 Y_{t-2} + Z_t, \quad Z_t \sim \mathrm{WN}(0, \sigma^2)$$

Problem: Depends on two lags $\rightarrow$ not Markovian (needs memory)

State-Space Transformation

Define the **state vector** $\mathbf{X}_t = \begin{pmatrix} Y_{t-1} \\ Y_t \end{pmatrix}$.

Then:

$$\text{State equation: } \mathbf{X}_{t+1} = \underbrace{\begin{pmatrix} 0 & 1 \\ \phi_2 & \phi_1 \end{pmatrix}}_{F} \mathbf{X}_t + \underbrace{\begin{pmatrix} 0 \\ Z_{t+1} \end{pmatrix}}_{\mathbf{v}_t}$$

$$\text{Observation equation: } Y_t = \underbrace{\begin{pmatrix} 0 & 1 \end{pmatrix}}_{G} \mathbf{X}_t$$

Now it's **first-order Markov**: $\mathbf{X}_{t+1}$ depends only on $\mathbf{X}_t$!

General AR(p) Representation (Companion Form)

The Companion Matrix Trick

For AR(p): $Y_t = \phi_1 Y_{t-1} + \cdots + \phi_p Y_{t-p} + Z_t$

- State vector: $\mathbf{X}_t = \begin{pmatrix} Y_{t-p+1} & Y_{t-p+2} & \cdots & Y_t \end{pmatrix}'$
- Contains exactly the **past** p **values** needed for prediction

Matrix Formulation

$$\mathbf{X}_{t+1} = \underbrace{\begin{pmatrix} 0 & 1 & 0 & \cdots & 0 \\ 0 & 0 & 1 & \cdots & 0 \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & 0 & \cdots & 1 \\ \phi_p & \phi_{p-1} & \phi_{p-2} & \cdots & \phi_1 \end{pmatrix}}_{\text{Companion matrix } F} \mathbf{X}_t + \underbrace{\begin{pmatrix} 0 \\ 0 \\ \vdots \\ 0 \\ Z_{t+1} \end{pmatrix}}_{\mathbf{v}_t}$$

$$Y_t = \underbrace{\begin{pmatrix} 0 & 0 & \cdots & 0 & 1 \end{pmatrix}}_{G} \mathbf{X}_t$$

Why This Works

The companion matrix F **shifts** the state and applies the AR coefficients in the last row. This is the standard way to turn scalar AR(p) into vector AR(1).

From ARMA to State-Space: The Two-Step Trick

ARMA(p,q) Challenge

$$Y_t = \phi_1 Y_{t-1} + \cdots + \phi_p Y_{t-p} + Z_t + \theta_1 Z_{t-1} + \cdots + \theta_q Z_{t-q}$$

Problem: Depends on both past Y's and past Z's.

Clever Representation

Let $r = \max(p, q+1)$. Define an AR(r) process $\{U_t\}$:

$$U_t = \phi_1 U_{t-1} + \cdots + \phi_r U_{t-r} + Z_t \quad (\text{set } \phi_j = 0 \text{ for } j > p)$$

Then Y_t is a **linear combination** of U_t and its lags:

$$Y_t = U_t + \theta_1 U_{t-1} + \cdots + \theta_{r-1} U_{t-r+1}$$

The Payoff

Now we have:

1. State equation: AR(r) for U_t (companion form as before)
2. Observation equation: $Y_t = G\mathbf{X}_t$ where $\mathbf{X}_t$ contains $U_{t-r+1}, \ldots, U_t$

We've converted ARMA to state-space without increasing state dimension too much!

Notation: ∇ is the Difference Operator

Definition

$$\nabla Y_t = Y_t - Y_{t-1} \quad \text{(first difference)}$$
$$\nabla^d Y_t = \underbrace{\nabla \cdots \nabla}_{d \text{ times}} Y_t \quad \text{(d-th order difference)}$$

Example: ARIMA(1,1,1)

$$(1-\phi B)(1-B)Y_t = (1+\theta B)Z_t$$

is equivalent to:

$$(1-\phi B)\underbrace{(Y_t - Y_{t-1})}_{\nabla Y_t} = (1+\theta B)Z_t$$

So ∇Y_t follows an ARMA(1,1) model.

In Words

ARIMA(p,d,q) = Apply d differences to Y_t, then model the result as ARMA(p,q).

Handling Non-Stationarity: ARIMA Models

The Differencing Challenge

ARIMA(p,d,q): $\nabla^d Y_t$ is ARMA(p,q)
But $\nabla^d Y_t = (1-B)^d Y_t$ loses the **levels** information.

State-Space Solution for ARIMA(1,1,1)

For $(1-\phi B)(1-B)Y_t = (1+\theta B)Z_t$:

- Let $X_t = \nabla Y_t - \theta X_{t-1}$ (a trick from the book)
- State vector: $\mathbf{X}_t = \begin{pmatrix} X_{t-1} & X_t & Y_{t-1} \end{pmatrix}'$
- State equation tracks both differences and levels
- Observation: $Y_t = \begin{pmatrix} \theta & 1 & 1 \end{pmatrix} \mathbf{X}_t$

General Principle

For ARIMA(p,d,q):

1. Put $\nabla^d Y_t$ in state-space (as ARMA)
2. Augment state with **past levels** $Y_{t-1}, \ldots, Y_{t-d}$
3. The observation equation reconstructs Y_t from differenced series and past levels

Why Bother? Practical Benefits

For Commodity Price Modeling

- **Unified framework**: Same algorithm (Kalman filter) for AR, MA, ARMA, ARIMA
- **Missing data**: Handle irregular observations in price series
- **Real-time updating**: As new prices arrive, update forecasts efficiently
- **Parameter estimation**: Compute exact likelihood for MLE
- **Model comparison**: Compare AR, MA, ARMA models in same framework

Quick Recap

Model Type	State Vector Contains
AR(p)	Last p observations $Y_{t-p+1}, \ldots, Y_t$
MA(q)	Last q innovations $Z_{t-q+1}, \ldots, Z_t$
ARMA(p,q)	Intermediate AR(r) process U_t and its lags
ARIMA(p,d,q)	Differences $\nabla^d Y_t$ AND past levels $Y_{t-d}, \ldots, Y_{t-1}$

9.4.1 Prediction, Filtering, and Smoothing Problems

Definition 9.4.1 (Best Linear Predictor)

For state $\mathbf{X}$, we seek the projection $P_t(\mathbf{X})$ onto observations $\mathbf{Y}_0, \ldots, \mathbf{Y}_t$.

Three Fundamental Problems

1. **Prediction**: Estimate future $\mathbf{X}_{t+h}$ given data up to t.
2. **Filtering**: Estimate current $\mathbf{X}_t$ given data up to t (removes noise).
3. **Smoothing**: Estimate past $\mathbf{X}_t$ given a full sample up to $n > t$.

Kalman Prediction Equations (9.4.1-9.4.2)

What we want to compute

At each time t, we aim to predict the next state $\mathbf{X}_{t+1}$ given observations up to time t:

$$\hat{\mathbf{X}}_{t+1} = P_t(\mathbf{X}_{t+1}) \quad \text{and} \quad \Omega_{t+1} = \mathrm{Cov}(\mathbf{X}_{t+1} - \hat{\mathbf{X}}_{t+1}).$$

Kalman Prediction Recursions

The Kalman filter provides recursive formulas:

$$\hat{\mathbf{X}}_{t+1} = F_t \hat{\mathbf{X}}_t + \underbrace{F_t \Omega_t G_t' \Delta_t^{-1}}_{\text{Kalman gain } K_t} \underbrace{(\mathbf{Y}_t - G_t \hat{\mathbf{X}}_t)}_{\text{innovation } \mathbf{I}_t} \tag{9.4.1}$$

$$\Omega_{t+1} = F_t \Omega_t F_t' + Q_t - \underbrace{F_t \Omega_t G_t' \Delta_t^{-1} G_t \Omega_t F_t'}_{\text{reduction due to new observation}} \tag{9.4.2}$$

where

$$\Delta_t = G_t \Omega_t G_t' + R_t, \qquad \Theta_t = F_t \Omega_t G_t'.$$

Multi-Step Prediction and the Innovations Approach

Interpretation

- $\mathbf{I}_t = \mathbf{Y}_t - G_t\hat{\mathbf{X}}_t$ is the **innovation** (the part of $\mathbf{Y}_t$ not predicted from the past).
- The Kalman gain $K_t = \Theta_t \Delta_t^{-1}$ tells us how much to adjust the state prediction based on the innovation.
- Equation (9.4.2) says: prediction uncertainty = (previous uncertainty propagated) + (new noise) - (reduction from observing $\mathbf{Y}_t$).

Why Innovations?

The sequence of innovations $\{\mathbf{I}_t\}$ is **orthogonal** (uncorrelated over time). This lets us decompose the projection:

$$P_t(\cdot) = P_{t-1}(\cdot) + P(\cdot \mid \mathbf{I}_t) \tag{9.4.3}$$

i.e., the best predictor given all data up to t equals the best predictor up to $t-1$ plus the contribution from the latest innovation.

h-Step Ahead Prediction

To predict $\mathbf{X}_{t+h}$ or $\mathbf{Y}_{t+h}$ given data up to t, we iterate the state equation:

$$P_t\mathbf{X}_{t+h} = (F_{t+h-1}F_{t+h-2}\cdots F_{t+1})P_t\mathbf{X}_{t+1}, \quad h \geq 2 \tag{9.4.6}$$

$$P_t\mathbf{Y}_{t+h} = G_{t+h}P_t\mathbf{X}_{t+h} \tag{9.4.7}$$

The corresponding error covariance matrices evolve as:

$$\Omega_t^{(h)} = F_{t+h-1}\Omega_t^{(h-1)}F'_{t+h-1} + Q_{t+h-1}, \quad h \geq 2 \tag{9.4.8}$$

$$\Delta_t^{(h)} = G_{t+h}\Omega_t^{(h)}G'_{t+h} + R_{t+h} \tag{9.4.9}$$

with initial conditions $\Omega_t^{(1)} = \Omega_{t+1}$ and $\Delta_t^{(1)} = \Delta_{t+1}$.

Practical Use

These formulas allow us to compute forecasts and their uncertainties for any horizon h recursively, using only the one-step Kalman predictions.

Kalman Filtering: Estimating the Current State

Filtering vs. Prediction

- **Prediction** (previous slide): Estimate $\mathbf{X}_{t+1}$ given data up to time $t \to \hat{\mathbf{X}}_{t+1}$
- **Filtering**: Estimate $\mathbf{X}_t$ given data up to time $t \to \mathbf{X}_{t|t}$

Filtering uses the *current* observation $\mathbf{Y}_t$ to improve the estimate of the *current* state.

Filtering Equations

After predicting $\hat{\mathbf{X}}_t$ (which uses data up to $t-1$), we incorporate the new observation $\mathbf{Y}_t$:

$$\mathbf{X}_{t|t} = \hat{\mathbf{X}}_t + \underbrace{\Omega_t G_t' \Delta_t^{-1}}_{\text{filter gain}} (\mathbf{Y}_t - G_t \hat{\mathbf{X}}_t) \tag{9.4.16}$$

$$\Omega_{t|t} = \Omega_t - \Omega_t G_t' \Delta_t^{-1} G_t \Omega_t \tag{9.4.17}$$

Interpretation

- The filter gain $\Omega_t G_t' \Delta_t^{-1}$ tells us how much to adjust the predicted state based on the innovation $\mathbf{Y}_t - G_t \hat{\mathbf{X}}_t$.
- Filtering is used for real-time applications (e.g., tracking a moving object, estimating current volatility).

Kalman Smoothing: Re-Estimating Past States

Smoothing vs. Filtering

- **Filtering**: Uses data up to time t to estimate state at time t.
- **Smoothing**: Uses data up to time n (with $n > t$) to estimate state at time t.

Smoothing gives a *better* estimate of past states because it uses future information.

Fixed-Point Smoothing Equations

Suppose we have data up to time n and want to improve the estimate of an earlier state $\mathbf{X}_t$ (for fixed t). The smoother updates recursively as n increases:

$$P_n \mathbf{X}_t = P_{n-1}\mathbf{X}_t + \underbrace{\Omega_{t,n} G_n' \Delta_n^{-1}}_{\text{smoothing gain}} (\mathbf{Y}_n - G_n \hat{\mathbf{X}}_n) \tag{9.4.19}$$

$$\Omega_{t,n+1} = \Omega_{t,n}[F_n - \Theta_n \Delta_n^{-1} G_n]' \tag{9.4.20}$$

$$\Omega_{t|n} = \Omega_{t|n-1} - \Omega_{t,n} G_n' \Delta_n^{-1} G_n \Omega_{t,n}' \tag{9.4.21}$$

When to Use Smoothing

- **Historical analysis**: Understanding what really happened in the past (e.g., reconstructing a trend or seasonal component).
- Smoothing can provide optimal estimates of missing values.

9.5.1 The Likelihood Function

The Problem

We want to estimate parameters θ (which include F_t, G_t, Q_t, R_t, etc.) from observed data $\mathbf{Y}_1, \ldots, \mathbf{Y}_n$.

The Idea: Prediction Errors

The likelihood measures how well the model predicts the data. For state-space models, the one-step prediction errors (innovations) are:

$$\mathbf{I}_t = \mathbf{Y}_t - \hat{\mathbf{Y}}_t, \quad \text{where } \hat{\mathbf{Y}}_t = G_t \hat{\mathbf{X}}_t$$

and $\hat{\mathbf{X}}_t$ is the predicted state from the Kalman filter.

Gaussian Likelihood (9.5.2)

If we assume Gaussian errors, the log-likelihood is:

$$\ln L(\theta) = -\frac{nw}{2}\ln(2\pi) - \frac{1}{2}\sum_{t=1}^{n}\ln\det(\Delta_t) - \frac{1}{2}\sum_{t=1}^{n}\mathbf{I}_t'\Delta_t^{-1}\mathbf{I}_t$$

where $\Delta_t = \mathrm{Cov}(\mathbf{I}_t)$ is the prediction error covariance from the Kalman filter.

How to Compute It

1. For a given θ, run the Kalman filter to get $\{\mathbf{I}_t, \Delta_t\}$.
2. Evaluate $\ln L(\theta)$ using the formula above.
3. Use an optimizer (e.g., BFGS) to find $\hat{\theta}$ that maximizes $\ln L(\theta)$.

9.5.2 Fitting Structural Models Efficiently

Structural Model

$$Y_t = G\mathbf{X}_t + W_t, \qquad W_t \sim N(0, \sigma_w^2)$$
$$\mathbf{X}_{t+1} = F\mathbf{X}_t + \mathbf{V}_t, \qquad \mathbf{V}_t \sim N(0, Q)$$

Parameters: $\mu = \mathbf{X}_1$, Q, and σ_w^2.

Challenge

Maximizing the likelihood over all parameters is hard when the state dimension is large.

A Simpler Approach: Reduced Likelihood (9.5.12)

We can profile out μ and σ_w^2:

1. For a given Q, compute $\hat{\mu}(Q)$ and $\hat{\sigma}_w^2(Q)$ by simple formulas (using Kalman filter outputs).
2. Then maximize the *reduced likelihood* $\ell(Q)$ which only depends on Q.

This reduces the optimization dimension.

Takeaway

Even for complex structural models, the Kalman filter lets us compute the likelihood efficiently, and profiling simplifies the optimization.

Example 9.5.2: International Airline Passengers

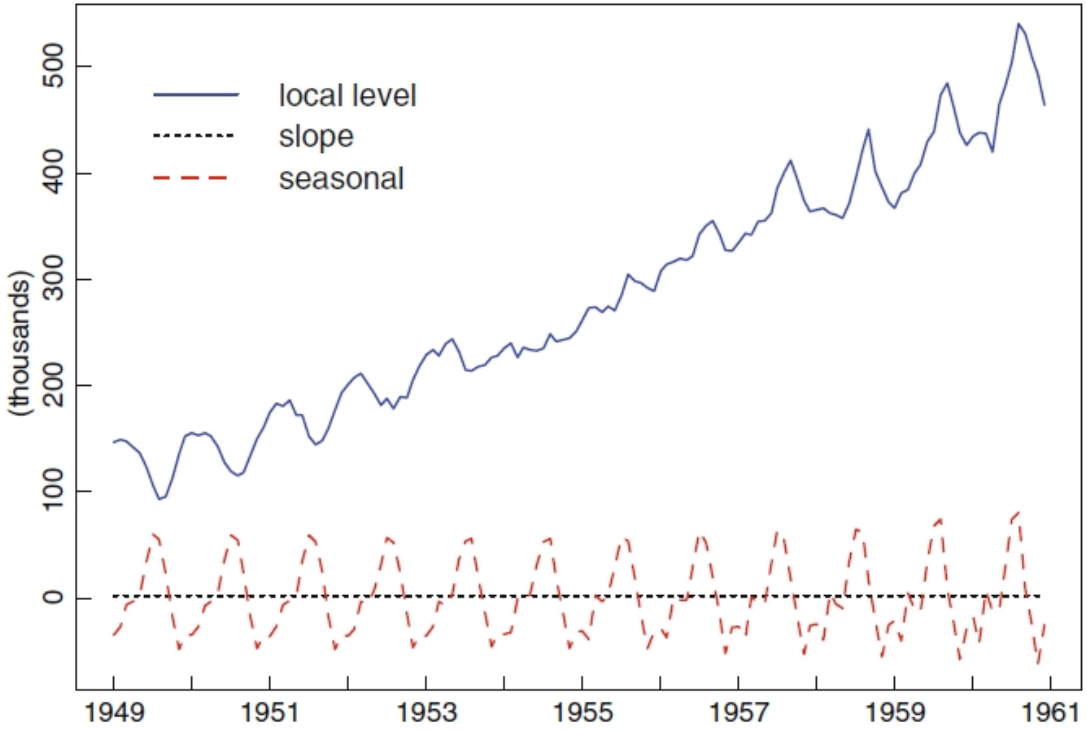


Figure: Figure 9-3: International airline passengers, 1949-1960.

Model Specification

Decomposes the series into trend, slope, and 11 seasonal factors ($d = 12$).

Airline Passengers Results

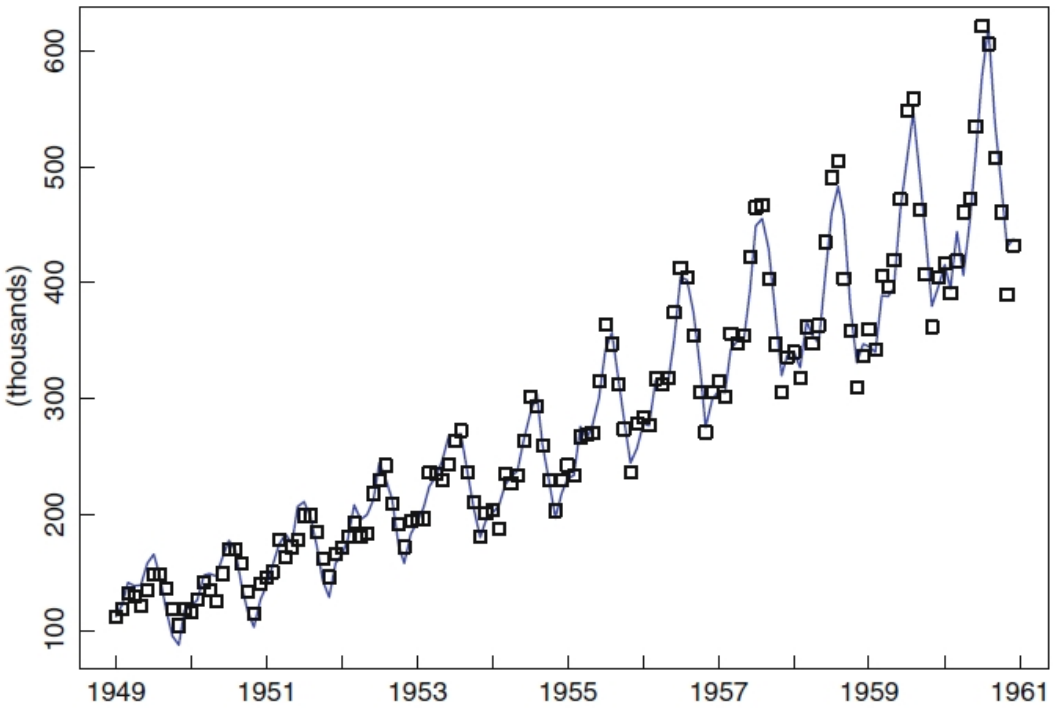


Figure: One-step predictors follow the data movement with high accuracy.

Parameter Estimates

- $\hat{\sigma}_2^2 \approx 0$ implies a very stable growth rate.
- Predictors provide excellent real-time decomposition.

9.6.1 Gaussian Likelihood with Missing Data

Modified Observation Equation (9.6.1-9.6.3)

Irregularly spaced observations $\mathbf{Y}_{i_1}, \ldots, \mathbf{Y}_{i_r}$:

$$\mathbf{Y}_t^* = G_t^* \mathbf{X}_t + \mathbf{W}_t^*, \quad G_t^* = 0 \text{ if missing.} \tag{9.6.1}$$

Key Insight

The Kalman filter naturally handles gaps by **skipping the update step**—propagating the state estimate purely via dynamics (F) until new data arrives.

Show how the Kalman filter handles missing observations

When an observation is missing, the filter:

1. **Skips the update step** (since there's no new data)
2. **Continues predicting** (state estimate uncertainty grows)
3. **Resumes updating** when data becomes available again

Concrete AR(1) Example

Model: $Y_t = \phi Y_{t-1} + Z_t$, $Z_t \sim N(0, \sigma^2)$
Data: We observe y_1, y_3, y_4, y_5 but y_2 is missing.
Without Kalman filter: Traditional approach requires complex likelihood computation.
With Kalman filter:

- At $t = 1$: Update with y_1 (normal)
- At $t = 2$: No observation $\rightarrow$ only prediction step
- $\hat{Y}_2 = \phi y_1$ (prediction based on y_1 only)
- Ω_2 (prediction variance) increases because no update
- At $t = 3$: Update with y_3, correcting the prediction

Key Insight for Commodity Data

Commodity price series often have gaps (holidays, illiquid contracts).
The Kalman filter handles these gaps **automatically** and **optimally**.

9.7.1 Expectation-Maximization (EM) Algorithm

The Problem: Maximum Likelihood with Missing Data

Suppose we have a model with parameters θ, observed data $\mathbf{Y}$, and unobserved (latent) data $\mathbf{X}$. The likelihood $L(\theta; \mathbf{Y})$ is hard to maximize directly because it involves integrating over $\mathbf{X}$. But if we could observe $\mathbf{X}$, the **complete-data likelihood** $L(\theta; \mathbf{X}, \mathbf{Y})$ would be easier to maximize.

The EM Idea: Two Steps That Guarantee Improvement

1. **E-step (Expectation)**: Given current parameter estimate $\theta^{(i)}$, compute the expected complete-data log-likelihood:

$$Q(\theta|\theta^{(i)}) = E_{\theta^{(i)}}[\ln f(\mathbf{X}, \mathbf{Y}; \theta)|\mathbf{Y}]$$

   This expectation is taken over the conditional distribution of $\mathbf{X}$ given $\mathbf{Y}$ with $\theta = \theta^{(i)}$.

2. **M-step (Maximization)**: Find the θ that maximizes $Q(\theta|\theta^{(i)})$:

$$\theta^{(i+1)} = \arg\max_{\theta} Q(\theta|\theta^{(i)})$$

Each iteration **never decreases** the observed-data likelihood: $L(\theta^{(i+1)}; \mathbf{Y}) \geq L(\theta^{(i)}; \mathbf{Y})$.

Complete Data = Observed $\mathbf{Y}$ + Latent States $\mathbf{X}$

In state-space models:

- $\mathbf{Y} = (\mathbf{Y}_1, \ldots, \mathbf{Y}_n)$: observed time series
- $\mathbf{X} = (\mathbf{X}_1, \ldots, \mathbf{X}_n)$: unobserved state vectors
- Complete-data likelihood $f(\mathbf{X}, \mathbf{Y}; \theta)$ factors nicely due to Markov structure

E-Step: Kalman Smoothing

Given $\theta^{(i)}$, we need $E_{\theta^{(i)}}[\mathbf{X}_t|\mathbf{Y}]$ and covariances for all t.

- Run **Kalman filter** forward to get $P(\mathbf{X}_t|\mathbf{Y}_1, \ldots, \mathbf{Y}_t)$
- Run **Kalman smoother** backward to get $P(\mathbf{X}_t|\mathbf{Y}_1, \ldots, \mathbf{Y}_n)$ (all data)
- For linear Gaussian models: these conditional distributions are normal with closed-form means and variances
- For non-Gaussian models: use approximations or Monte Carlo methods

M-Step: "As If Complete Data"

Maximize $Q(\theta|\theta^{(i)}) = E_{\theta^{(i)}}[\ln f(\mathbf{X}, \mathbf{Y}; \theta)|\mathbf{Y}]$.

- For linear Gaussian models: update F, G, Q, R using sample covariances of smoothed states
- For ARMA models: use standard MLE formulas on the "complete" series

Why EM Works Well for State-Space Models

Theoretical Justification

- **Monotonicity**: Each EM iteration increases the likelihood
- **Convergence**: Under mild conditions, converges to a local maximum
- **Stability**: More stable than direct gradient methods for missing data

Practical Advantages

- **Modular**: E-step uses existing Kalman filter/smoother code
- **Interpretable**: M-step often has closed form (e.g., regression on smoothed states)
- **Efficient**: For models where complete-data MLE is simple

Example 9.7.1: Lake Data with Missing Values

The Problem: Fitting AR(2) to Incomplete Data

We have the **mean-corrected lake data** $\{W_t\}$ but with 10 missing values at times $t = 17, 24, 31, 38, 45, 52, 59, 66, 73, 80$.

- **Goal**: Fit AR(2) model $W_t = \phi_1 W_{t-1} + \phi_2 W_{t-2} + Z_t$ to this incomplete dataset.
- **Traditional approach**: Complex likelihood computation with integrals over missing values.
- **EM approach**: Iteratively estimate missing values and AR parameters.

Lake Data (cont.)

Step-by-Step EM Procedure

1. **Initialization**: Start with initial parameter guess $\phi_1^{(0)} = \phi_2^{(0)} = 0$ (white noise).
2. **E-step (Iteration 1)**: Given current parameters $\phi^{(i)}$, estimate missing values:
   - Use **Kalman smoother** to compute $E[W_t|\text{all observed data}]$ for each missing t.
   - For initial $\phi^{(0)} = (0, 0)$, model is white noise $\rightarrow$ missing value estimates $= 0$.
3. **M-step (Iteration 1)**: Fit AR(2) to "complete" dataset (observed values + estimated missing values):
   - Use standard AR(2) MLE formulas on the filled-in series.
   - Obtain $\phi^{(1)} = (1.0729, -0.2838)$.
4. **Repeat**: Update missing value estimates using $\phi^{(1)}$, then re-estimate parameters.
5. **Convergence**: After 4 iterations, parameters stabilize at $\hat{\phi}_1 = 1.0999$, $\hat{\phi}_2 = -0.3128$.

Iteration	W_{17}	W_{24}	W_{31}	ϕ_1	ϕ_2
0	0.000	0.000	0.000	0.0000	0.0000
1	0.534	0.205	0.746	1.0729	-0.2838
2	0.458	0.393	0.821	1.0999	-0.3128
3	0.454	0.405	0.826	1.0999	-0.3128
4	0.454	0.405	0.826	1.0999	-0.3128

Table: Table 9.1: EM algorithm convergence. Missing value estimates stabilize as parameters converge.

Why This Matters for Commodity Markets

- Commodity data often has **gaps** (holidays, trading halts, illiquid contracts).
- EM allows **exact maximum likelihood estimation** despite missing data.
- The Kalman smoother provides **optimal estimates** of missing values given current parameters.
- Final parameter estimates are **consistent** with complete-data MLE.

2.3.3 Application to Commodity Forward Curves

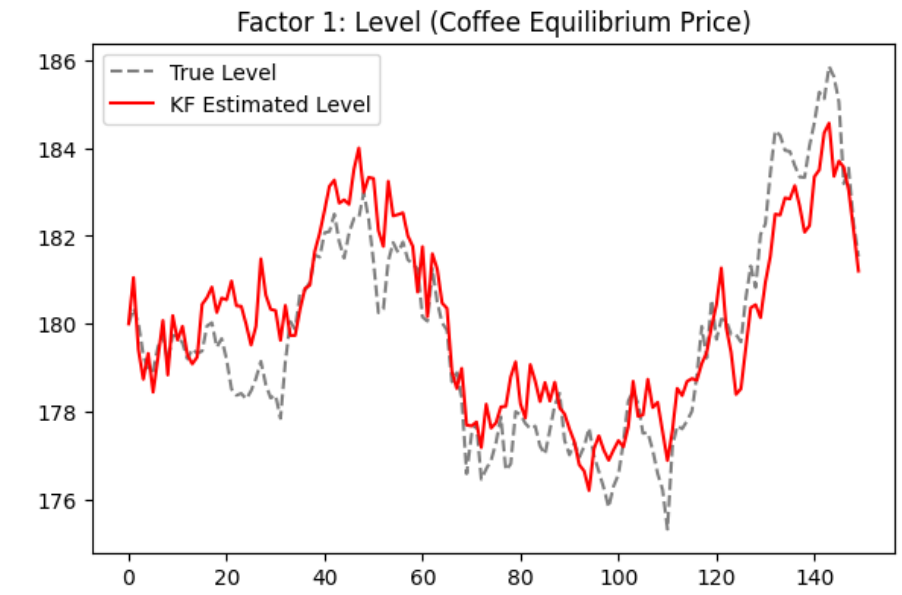
Modeling Framework

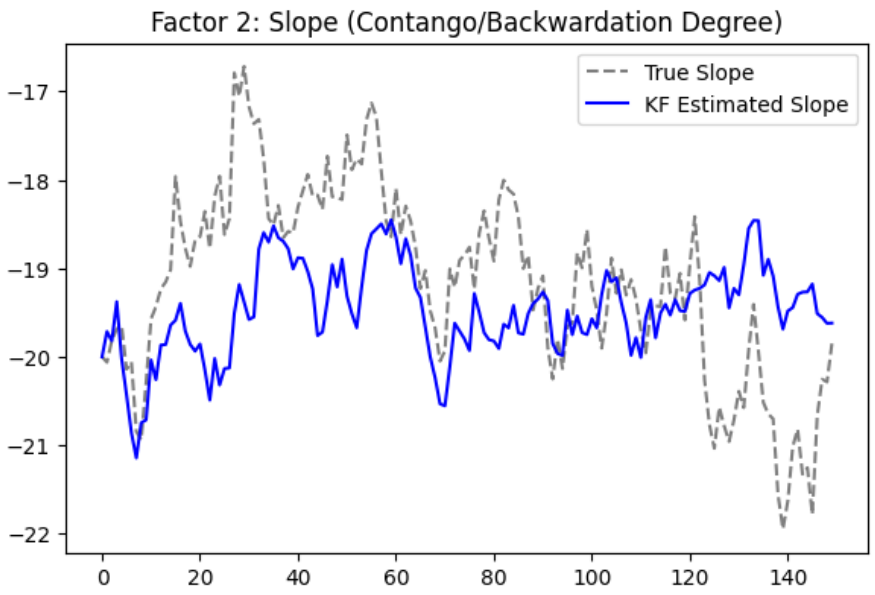
- **States**: Latent factors (Level, Slope, Curvature).
- **Observations**: Noisy futures prices at different maturities.

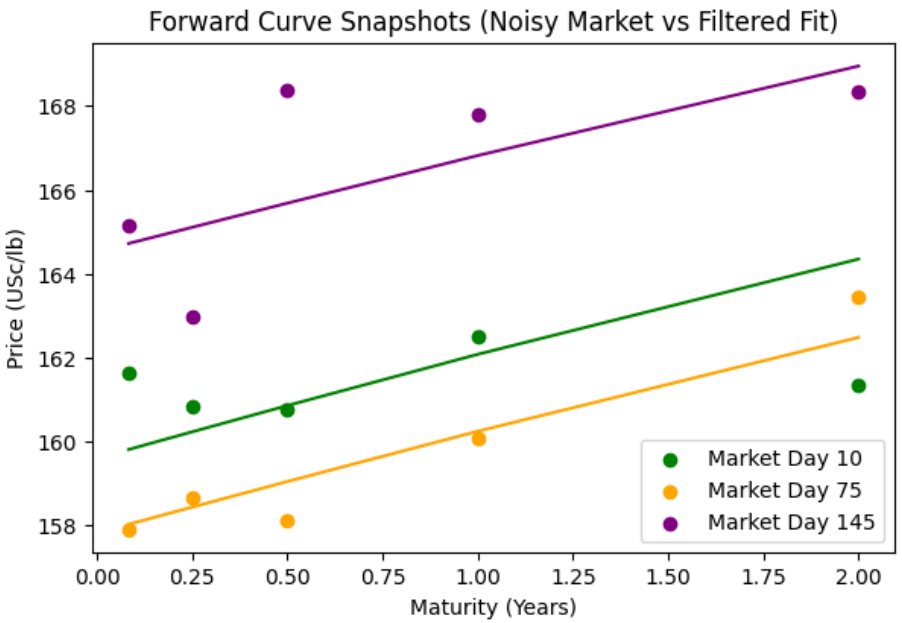
Estimation Procedure

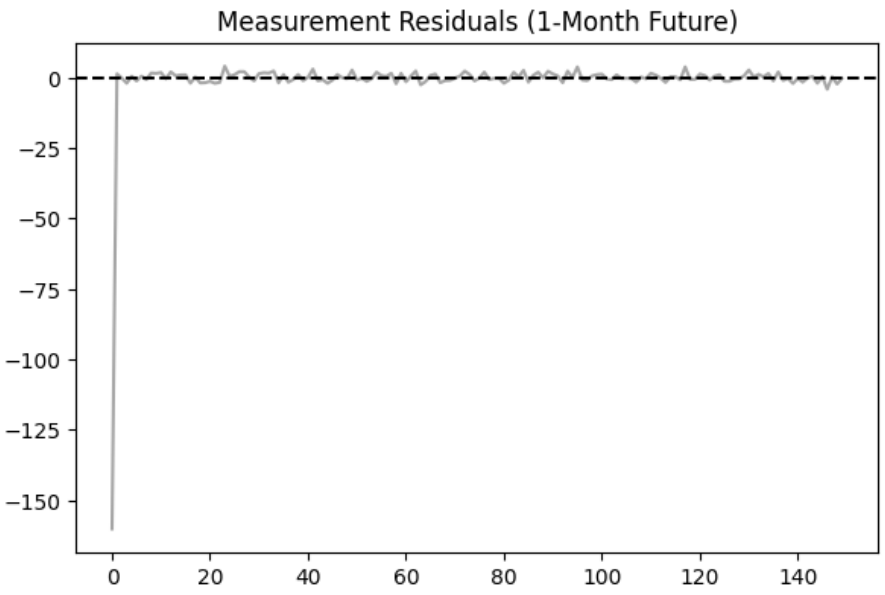
1. Specify state-space form (e.g., Dynamic Nelson-Siegel).
2. Estimate parameters via MLE.
3. Extract latent curve factors using the Kalman Filter.

Extracting the "Signal" from Coffee Market Noise









1. **State Tracking (Top Panels)**
   - The **Dashed Gray Line** is the "Hidden Reality" (Simulated).
   - The **Solid Colored Line** is what the KF "believes" reality is.
   - Note the *Initialization Phase*: The filter starts with a guess and rapidly converges as it "learns" from the incoming data.
2. **Denoising the Curve (Bottom Left)**
   - **Dots**: Noisy market quotes for different maturities.
   - **Lines**: The smoothed Nelson-Siegel curve.
   - **Pedagogical Value**: Even when market data is messy/sparse, the KF provides a continuous, arbitrage-free curve for pricing.
3. **Residual Diagnostics (Bottom Right)**
   - This plot tracks the **Innovations** $(\mathbf{Y}_t - G\hat{\mathbf{X}}_t)$.
   - Ideally, this should look like "white noise" centered at zero.
   - *Insight*: The large spike at $t = 0$ shows the filter correcting its initial uncertainty. Stable residuals after $t = 20$ indicate the model is well-calibrated.

The "Trader's" Takeaway

In the real world, you never see the gray dashed line. The Kalman Filter is your "glasses"—it turns the scattered dots into a actionable trade signal for the Level (Price) and Slope (Contango).

Summary and Key Takeaways

1. State-Space Representation: A Unified Framework

- **Observation equation**: $\mathbf{Y}_t = G_t \mathbf{X}_t + \mathbf{W}_t$ (what we observe)
- **State equation**: $\mathbf{X}_{t+1} = F_t \mathbf{X}_t + \mathbf{V}_t$ (how the hidden state evolves)
- **Key idea**: Separate the latent, unobservable state $\mathbf{X}_t$ from observation noise.
- **Flexibility**: Can represent ARIMA, structural models, regression with time-varying coefficients, etc.

2. The Kalman Filter: Optimal Recursive Estimation

- **Prediction**: $\hat{\mathbf{X}}_{t+1} = F_t \hat{\mathbf{X}}_t + K_t(\mathbf{Y}_t - G_t \hat{\mathbf{X}}_t)$
- **Filtering**: $\mathbf{X}_{t|t} = \hat{\mathbf{X}}_t + \Omega_t G_t' \Delta_t^{-1}(\mathbf{Y}_t - G_t \hat{\mathbf{X}}_t)$
- **Smoothing**: Re-estimate past states using future data (fixed-point smoothing)
- **Optimality**: Minimum mean squared error (MMSE) estimates for linear Gaussian models.

3. Practical Benefits for Time Series Analysis

- **Missing data**: Handles irregular observations naturally (skip update when missing).
- **Parameter estimation**: Maximize Gaussian likelihood via prediction error decomposition.
- **Forecasting**: Multi-step forecasts with uncertainty quantification.
- **Non-Gaussian data**: Generalized state-space models (e.g., Poisson, exponential family).

4. Applications in Commodity Markets

- **Forward curve modeling**: Latent factors (level, slope, curvature) with state-space dynamics.
- **Structural models**: Stochastic trend, seasonality, and cycles for price decomposition.
- **Real-time updating**: As new prices arrive, update forecasts and volatility estimates.
- **Handling illiquidity**: Smooth missing observations in less liquid contracts.

Remember

State-space models and the Kalman filter provide a **systematic, recursive, and optimal** way to estimate hidden states and forecast time series, even with *missing* data!